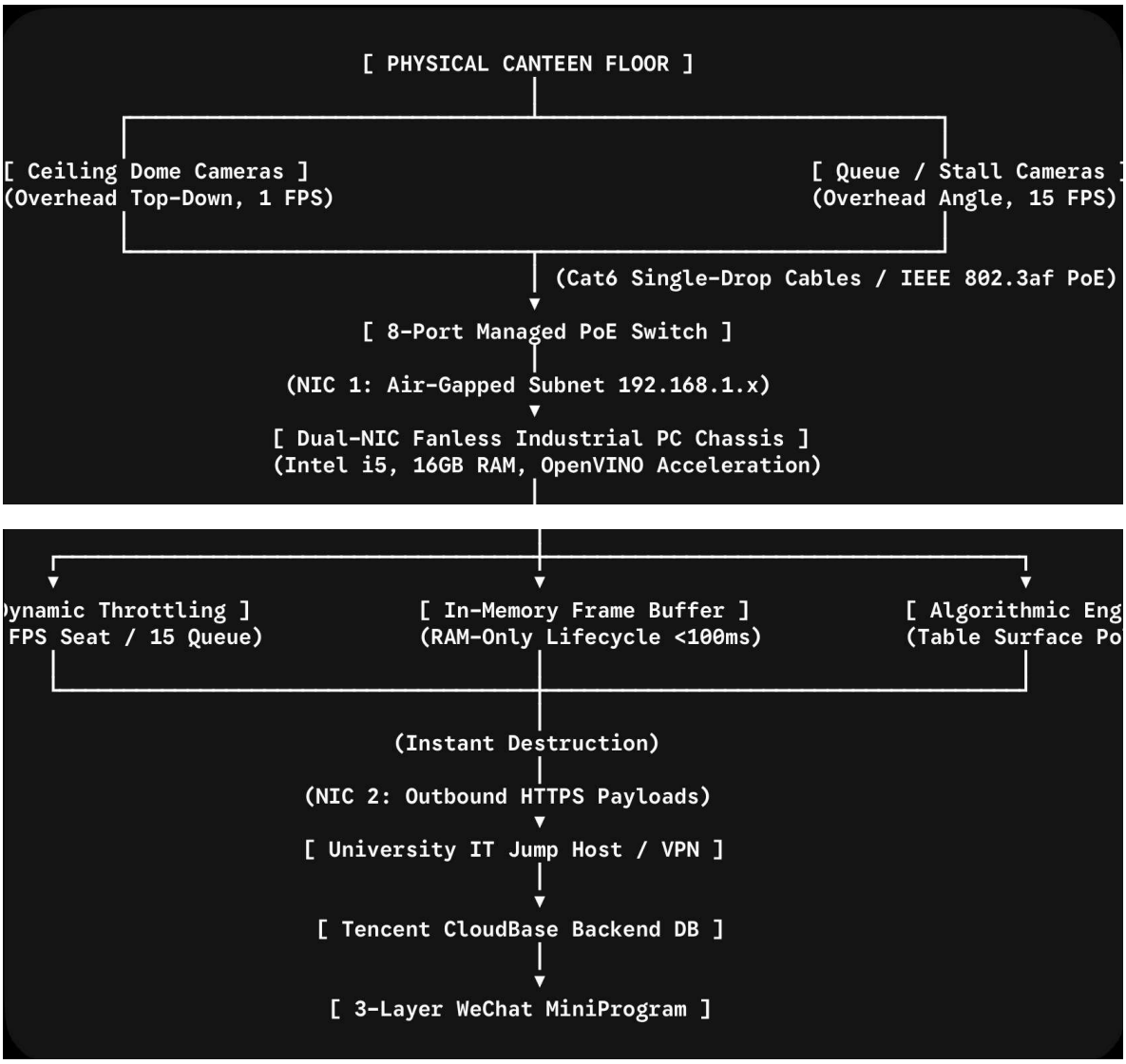
# THUCan Exhaustive Technical & Operational Specification Master Document

This document records **every single engineering decision, trade-off, legal resolution, cost re-evaluation, and product refinement.**

## Executive Architecture Overview




(overhead changed to angled view)

## 1. Complete Changelog

| Feature | Final Master Specification | Rationale & Engineering Advantage |
|---|---|---|

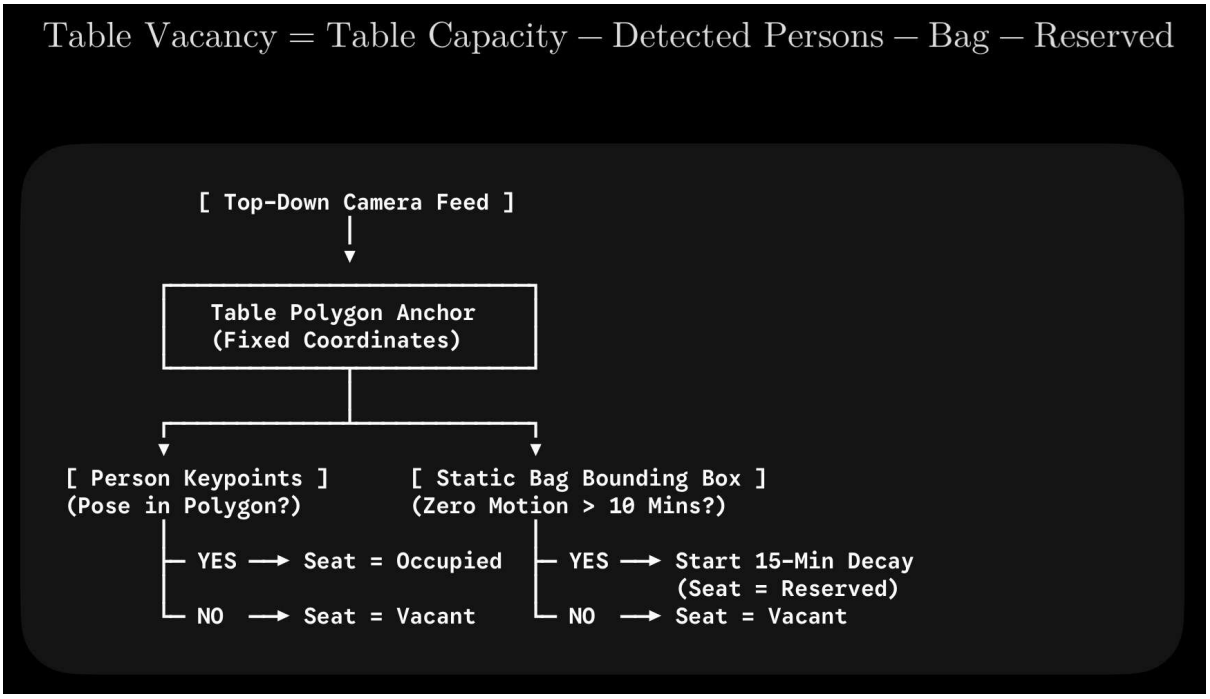
| | | |
|---|---|---|
| **Camera Hardware** | Commercial Ceiling-Mounted **Dome PoE IP** Cameras (IEEE 802.3af). | Eliminates 2.4GHz Wi-Fi congestion; dome housing protects lenses from ambient steam. |
| **Edge Host PC** | **Sealed Aluminum** Fanless Industrial **Chassis** (Intel i5, Dual-NIC). | Prevents vaporized kitchen oil and dust from clogging cooling fans and causing thermal failure. |
| **Network Architecture** | Air-Gapped **Dual-NIC** Setup (NIC 1: Isolated 192.168.1.x; NIC 2: Campus VPN). | Ensures video streams physically cannot leak outside the local switch. Replaces third-party Tailscale with university IT-managed VPN. |
| **Inference Rate** | Dynamic Frame Throttling (1 FPS Seating / 15 FPS Queue) + 30s Smoothing Filter. | Keeps iGPU utilization below 36%; 30s temporal smoothing prevents UI flickering from aisle walking. |
| **Detection Method** | Table-Surface **Polygon Anchoring** + Fallback **Pose Evaluation**. | Anchors tracking to immovable tables; chairs frequently shift, causing false positives. |
| **Open Source Model** | RT-DETR / **YOLOX** or Permissive Licensed YOLO Engine. | Addresses AGPL licensing concerns for university ownership. |
| **Data & Privacy Policy** | **RAM-Only Processing** (<100ms) + PIPIA Audit + Entrance Signage. | Fully satisfies PIPL Articles 26 & 28; transforms video input into anonymous JSON counts. |
| **Entrance Tracking** | Secondary Macro-Validator with Nightly Auto-Reset (10:00 PM). | Line-crossing accumulates drift during crowd surges; auto-resets nightly and calibrates against the sum of active table counts. |
| **Coat/Bag Detection** | Motion-Vector Delta + 15-Minute Decay Clock. | If an item shows zero frame-to-frame pixel motion for 10 minutes, a 15-minute clock converts status to `Unverified / Likely Abandoned`. |
| **Lens Maintenance** | Nightly **Automated Laplacian Variance Script** + Bi-Weekly Physical Wipe Schedule. | Automatically flags blurred or greasy lenses on maintenance dashboards. |

| | | |
|---|---|---|
| **Fail-Safe UI** | Automated **Heartbeat** (1-Min Pulse) + Zone Gray-Out (`Data Unavailable`). | Displays clear system status if hardware fails, preventing inaccurate readings. |
| **Navigation & Map** | Embedded 2D Vector Map with **Color Clusters** (Green/Yellow/Red). | Eliminates app-switching friction; presents intuitive visual density heatmaps. |
| **Social / Group Filter** | Group Seating Finder (`Solo`, `Pair`, `Group of 4+`). | Highlights table clusters with sufficient adjacent vacant seats. |
| **Menu & Stall Data** | Centralized University Catering Database + 1-Tap Merchant Toggle with Auto-Expiry. | Centralized menu updates from university feeds; auto-expiring badges prevent stale "Sold Out" flags. |
| **Rating system** | Student based ratings on each food item per stall | Ratings restricted to users whose WeChat location verifies they are physically near the canteen. One dish rating per user every 12 hours . Outlier ratings (spamming) are filtered out using trimmed mean algorithms. |
| **Crowd Forecasting** | Class-Timetable Heuristics Phase - Learned Time-Series Model. | Starts with class schedule heuristics before transitioning to historical data models. |
| **Bilingual UI** | Bilingual Chinese / English Interface. | Serves international students, scholars, and exchange visitors. |
| **Institutional Framing** | Cross-Departmental Alignment (Logistics + Security + IT). | Positioned as an operational utility managed by Logistics, supported by IT networks, and reviewed by Security. |
| **Budget & Costing** | Detailed CapEx (¥5,500/floor) & OpEx (¥1,000/yr) Budget Breakdown. | Provides a realistic cost breakdown (~¥30–¥35/seat) for university facilities review. |

## 2. Granular Technical & Algorithmic Specifications

### A. The Table Vacancy Calculation Formula

The edge computer evaluates table status using a multi-factor equation:




- **Table Capacity** → Fixed integer defined during calibration (e.g., Table 12 = 4 seats).
- **Detected Persons** → Number of human head/shoulder keypoints detected within the tabletop boundary.
- **Bag-Reserved Seats** → Number of `backpack`, `handbag`, `coat`, or `laptop` bounding boxes overlapping a seat polygon without a human present.

## B. Sample Code for Seat Occupation

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>

enum class SeatState { VACANT, OCCUPIED, RESERVED, POTENTIALLY_ABANDONED };

struct TrackedObject {
    std::string label;
    float confidence;
};

class SeatEvaluator {
private:
    double unmoved_timer_sec = 0.0;
    double decay_timer_sec = 0.0;
```

```
    static constexpr double UNMOVED_THRESHOLD_SEC = 600.0; // 10 mins
    static constexpr double DECAY_THRESHOLD_SEC = 900.0;   // 15 mins
    static constexpr double MOTION_DELTA_THRESHOLD = 0.05;

public:
    SeatState evaluateState(const std::vector<TrackedObject>& detected_objects,
                            double motion_delta,
                            double frame_interval_sec)
    {
        // Check for person presence
        bool person_present = std::any_of(detected_objects.begin(), detected_objects.end(),
            [](const TrackedObject& obj) { return obj.label == "person"; });

        if (person_present) {
            unmoved_timer_sec = 0.0;
            decay_timer_sec = 0.0;
            return SeatState::OCCUPIED;
        }

        // Check for static belongings
        bool item_present = std::any_of(detected_objects.begin(), detected_objects.end(),
            [](const TrackedObject& obj) {
                return obj.label == "backpack" || obj.label == "jacket" || obj.label == "laptop";
            });

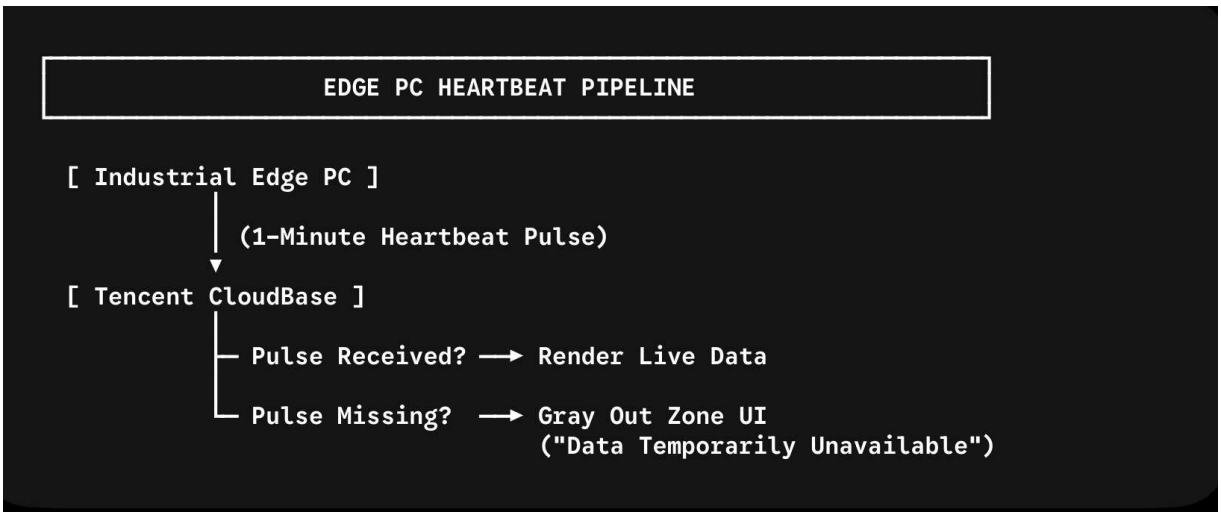
        if (item_present) {
            if (motion_delta < MOTION_DELTA_THRESHOLD) {
                unmoved_timer_sec += frame_interval_sec;
            } else {
                unmoved_timer_sec = 0.0; // Reset timer if item moved
            }

            if (unmoved_timer_sec > UNMOVED_THRESHOLD_SEC) {
                decay_timer_sec += frame_interval_sec;
                if (decay_timer_sec > DECAY_THRESHOLD_SEC) {
                    return SeatState::POTENTIALLY_ABANDONED;
                }
```

```
        }
        return SeatState::RESERVED;
    }

    // Neither person nor items detected
    unmoved_timer_sec = 0.0;
    decay_timer_sec = 0.0;
    return SeatState::VACANT;
  }
};
```

## C. System Resilience & Fail-Safe Mechanisms




1. **Heartbeat Signal** → The Mini PC sends an encrypted 1-minute pulse to Tencent CloudBase.
2. **Graceful Degradation** → If an edge PC loses power or camera links break, the server marks the affected zone as `Data Temporarily Unavailable`.
3. **Data Staleness Badge** → All MiniProgram screens display a subtle timestamp indicator: *"Updated 12s ago."*

## D. Camera Setup Plan

### Hardware

| Hardware Component | Specification | Purpose |
| --- | --- | --- |
| **PoE Dome Cameras** | 8x Indoor IP65/IK10 1080p **Dome Cameras** (Wide-Angle, 2.8mm fixed lens). | Captures video feeds while resisting ambient steam and dust. |

| | | |
|---|---|---|
| **Cat6 Cabling** | Standard UTP **Cat6** Solid Copper **Ethernet Cables** ( plenum-rated if required). | Delivers simultaneous 1Gbps data and 48V PoE power up to 100 meters. |
| **PoE Network Switch** | 1x **8-Port** Managed **PoE+ Switch** (IEEE 802.3af/at compliant, 120W power budget). | Distributes power and aggregates camera feeds into a single local subnet. |
| **Edge Compute Host** | 1x **Sealed Fanless Industrial PC** (Intel Core i5, 16GB RAM, 256GB NVMe SSD, Dual-NIC). | Runs the OpenVINO AI model and polygon intersection logic silently without cooling fans. |
| **Power Backup (UPS)** | 1x Compact **Mini UPS** (500VA/300W output). | Keeps the switch and industrial PC alive during brief power sags or flickers. |
| **Mounting Hardware** | Articulated **ceiling/wall bracket mounts** + weather-sealed junction boxes. | Secures cameras firmly to concrete beams or drop-ceiling structural rails. |

## Camera Positioning

A. Where They Are Placed

- **Mounting Locations** → Perimeter concrete pillars, high wall beams, or structural ceiling suspension rails around the perimeter of the dining hall.
- **Mounting Height** → **3.5 to 4.5 meters** off the ground.
- **Look Angle** → **45 to 55 deg steep diagonal** looking down into the dining zones.

B. Why This Positioning Wins

1. **Beating the "Tall Student" Blockade** → If cameras were mounted horizontally on side walls at eye level, a tall student sitting in the front row would completely block the view of everyone behind them. A high-angle ceiling mount looks *down over* heads.
2. **Avoiding Floor-Traffic Clutter** → Mounting cameras above 3.5 meters keeps them safely out of reach of students, brooks, and food carts while providing an unobstructed line of sight straight to the tabletops.
3. **Built-In Privacy Protection** → Because the camera looks down at a diagonal, it captures the **backs of heads, shoulders, and jackets**, never faces. This naturally satisfies PIPL privacy requirements by ensuring no facial biometrics are ever recorded.
4. **Cross-Angle Dual Coverage** → To eliminate blind spots entirely, rows of tables are covered by **two opposing cameras**. Camera A looks down-right across a row, and Camera B looks down-left. A table

is marked occupied if *either* camera detects an anchor point, and vacant only when *both* cameras confirm it is empty.

### Step-by-Step Installation

Step 1: The Network Closet Setup

- Install a locked, wall-mounted steel utility cabinet in a back maintenance room or above a ceiling access panel.
- Mount the **Fanless Industrial PC**, the **8-Port Managed PoE Switch**, and the **Mini UPS** inside the cabinet.
- Connect **NIC 2** of the industrial PC to the campus intranet/VPN drop, and plug the switch into **NIC 1** to establish the isolated camera subnet (`192.168.1.x`).

Step 2: Cable Drops & Routing

- Run individual Cat6 cables from the utility cabinet through suspended ceiling trays or drop-ceiling tiles straight to each camera's designated pillar or beam.
- Terminate each cable with an RJ45 connector, leaving a clean service loop inside a sealed junction box at the mounting point.

Step 3: Physical Mounting

- Bolt the articulated dome camera bracket to the structural beam or ceiling tile backing plate.
- Snap the IP dome camera into the bracket, plug in the Cat6 cable, and lock the tamper-resistant housing ring.

Step 4: System Calibration & Polygon Mapping

- Power on the system. Access the industrial PC locally via terminal.
- Open the built-in calibration web utility, pull up the live RTSP stream from each camera, and click-to-draw the 2D polygon boundaries over each physical table (`Table_01_Polygon`, `Table_02_Polygon`, etc.).
- Save the coordinate matrices to local memory, initialize the OpenVINO model, and verify that real-time JSON payloads are transmitting to Tencent CloudBase.

```
from openvino.runtime import Core
import cv2

# 1. Load optimized model
ie = Core()
model = ie.read_model(model="yolov8n_openvino_model/yolov8n.xml")
compiled_model = ie.compile_model(model=model, device_name="GPU")

# 2. Run inference on a frame slice
results = compiled_model(frame)
# 3. Output gives bounding box coordinates [x1, y1, x2, y2, confidence, class_id]
```

### Operational Logistics & Engineering Realities

Successfully deploying hardware in a university canteen requires managing real-world environmental and administrative factors:

- **Facilities Coordination (后勤处)** → Installation must be coordinated with campus facilities management to access ceiling spaces and run cable trays without interfering with fire sprinkler lines or HVAC ductwork.
- **Night-Shift Deployment** → All physical drilling, ladder work, and cable pulling must take place between **10:00 PM and 6:00 AM** when the canteen is completely closed and empty.
- **Environmental Hardening** → Canteen air carries micro-droplets of cooking oil and steam. Using **IP65-rated dome enclosures** prevents grease buildup on internal lenses, while the **sealed aluminum fanless PC** ensures kitchen dust never clogs internal heatsinks.
- **Preventative Maintenance:**
    - *Physical* → A facilities worker wipes down camera domes with a microfiber cloth once every two weeks.
    - *Software* → The automated nightly **Laplacian Variance script** runs at 2:00 AM, checking image sharpness and flagging any camera experiencing lens degradation or grease occlusion on the morning maintenance dashboard.

## E. Crowd Forecasting Mechanism

### Core Data Philosophy

A common temptation is to try and track individual student schedules to predict where they will eat. **We explicitly avoid this** because it creates massive data privacy overhead and violates PIPL.

Instead, THUCan treats crowd forecasting as a macro-system problem driven by three non-personal variables:

1. **The Bell Schedule (University Timetable Metadata)** → Public knowledge of when lectures end across different teaching buildings.
2. **Historical Density Logs** → Past occupancy curves stored anonymously in Tencent CloudBase.
3. **Real-Time Rate of Change (dD/dt)** → How fast the canteen is filling up or clearing out *right now*.

### Phase 1: The Cold-Start Heuristic Model (Weeks 1–3)

Every university has a fixed academic calendar and daily class schedule (e.g., Morning classes end at 11:50 AM; Afternoon classes end at 5:40 PM).

- We map the geographic distance and walking time from major teaching buildings to nearby canteens.

## Phase 2: The Data Pipeline & Storage

As the system runs every day, the edge PC continuously logs anonymized density states to **Tencent CloudBase** every 30 seconds.

The Log Structure (Stored in CloudBase DB):


```
{
  "timestamp": "2026-06-06T12:05:00Z",
  "canteen_id": "zijing_canteen_fl1",
  "zone_id": "zone_b_north_tables",
  "occupancy_percentage": 82.5,
  "queue_wait_min": 7,
  "day_of_week": "Friday",
  "weather": "Sunny"
}
```


Every night at midnight, a cloud function aggregates these logs into historical time-series datasets, building the training foundation for the machine learning model.

## Phase 3: The Machine Learning Prediction Engine

Once 3 to 4 weeks of historical logs accumulate, we transition from raw heuristics to a **Lightweight Time-Series Regressor** (such as an **XGBoost Regressor** or **Facebook Prophet** model running as a background cloud function).

To predict density at t + 15 mins and t + 30 mins, the model evaluates:

1. **Lagged Occupancy** → What was the occupancy 30, 60, and 90 minutes ago?
2. **Temporal Features** → Hour of day, day of week, day of month.
3. **External Context** → Weather conditions (rain forces students to stay closer to indoor canteens; clear weather scatters them).
4. **Current Velocity (dD/dt)** → Is occupancy accelerating or decelerating right now?

```python
import xgboost as xgb
import pandas as pd

def predict_future_density(current_zone_logs, minutes_ahead=15):
    """
    Predicts zone occupancy percentage 15 or 30 minutes into the future.
    """
    model = xgb.Booster()
    model.load_model("canteen_crowd_model.json")

    # Extract current real-time features
    latest_log = current_zone_logs[-1]
    features = pd.DataFrame([{
        'hour': pd.to_datetime(latest_log['timestamp']).hour,
        'minute': pd.to_datetime(latest_log['timestamp']).minute,
        'day_of_week': latest_log['day_of_week_encoded'],
        'current_occupancy': latest_log['occupancy_percentage'],
        'velocity': latest_log['occupancy_percentage'] - current_zone_log
        'weather_condition': latest_log['weather_encoded']
    }])

    dmatrix = xgb.DMatrix(features)
    predicted_occupancy = model.predict(dmatrix)[0]

    return round(predicted_occupancy, 1)
```

## 3. Financial Analysis: Real-World CapEx & OpEx Breakdown

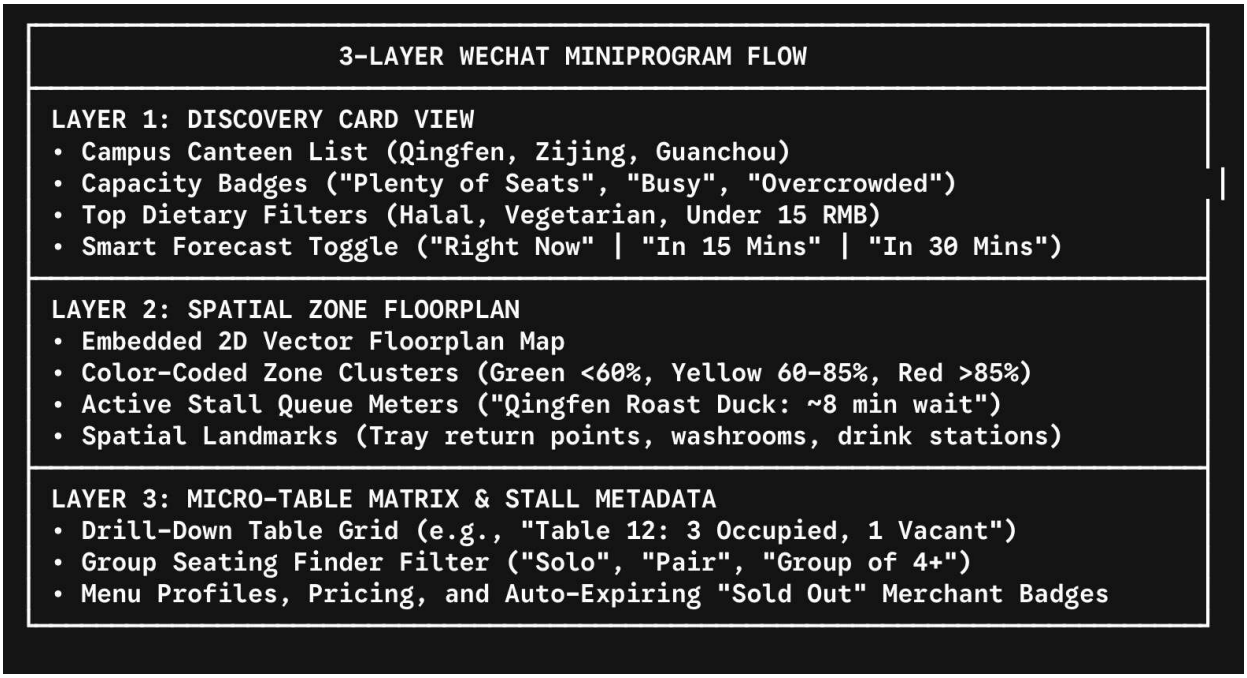
### Budget Model: 1 Canteen Floor (~150–200 Seats, ~30–40 Tables)

| Item | Model | Unit Cost | Quantity Required | Subtotal |
|---|---|---|---|---|
| **PoE Dome Cameras** | Commercial 1080p Dome IP Camera (IEEE 802.3af, Wide Dynamic Range) | ¥300 | ~ 8 units | ¥2,400 |
| **Industrial Edge Host** | Fanless Aluminum PC (Intel Core i5, 16GB RAM, Dual Gigabit NICs) | ¥2,200 | 1 unit | ¥2,200 |
| **Managed PoE Switch** | 8-Port Gigabit IEEE 802.3af Managed PoE Switch | ¥500 | 1 unit | ¥500 |
| **Structured Cabling** | Cat6 Plenum-rated Ethernet Cables, | ¥400 | Bulk Lot | ¥400 |

| | | | | |
|---|---|---|---|---|
| | Connectors, & Mounting Brackets | | | |
| **Capital Expenditure** | **Hardware Infrastructure Cost per Floor** | — | — | **¥5,500** |

- **Cost per Seat Ratio:** approx ¥34.38 RMB
- **Annual Operating Expenditure (OpEx):** Estimated at **¥1,000 RMB ($140 USD)/year** for bi-weekly lens cleaning supplies, maintenance buffer components, and cloud database utilities.

## 4. The 3-Layer WeChat MiniProgram Client Architecture




## 5. Judge & Panel Rehearsal Q&A Sheet

### Q1: "Isn't this surveillance?"

**Answer:** "No. The system functions as an anonymous IoT spatial sensor. Frames exist in volatile RAM for less than 100 milliseconds and are deleted instantly. No video or facial templates are ever recorded or saved. The output is a simple binary string like `Table 12: 1 seat free`. Furthermore, the camera network is physically air-gapped from the internet on its own local switch."

### Q2: "Why not use student card-swipe or Wi-Fi probe data instead?"

**Answer:** "Card swipes tell you how many people entered the building, but they can't tell you *where* the free seats are. A canteen can be at 60% capacity overall, but a student entering Zone A might find zero open

tables. Computer vision provides micro-location guidance—directing students to exact zones and tables with available seating."

### Q3: "What does it cost, and who pays?"

**Answer:** "Since this is an internal university utility, there are no commercial user fees or licensing models. The initial hardware setup costs roughly ¥5,500 per canteen floor—about ¥34 per seat. The project is funded by the University Logistics Office (后勤处) as part of campus smart-logistics infrastructure."

### Q4: "How accurate is it, and what happens when a camera gets blocked or greasy?"

**Answer:** "We use a 30-second temporal smoothing filter so people walking past tables don't trigger false readings, and state counts are aggregated to the table level for robustness. For maintenance, a nightly automated script checks image sharpness via Laplacian variance and alerts facilities staff if a lens needs wiping. If a camera fails completely, the app greys out that specific zone showing 'Data Unavailable' rather than displaying wrong counts."

### Q5: "Who updates menu items and 'Sold Out' status?"

**Answer:** "Menu and price data come directly from the official university catering database. For real-time updates during rush hours, canteen staff have a simple 1-tap merchant toggle on WeChat to mark dishes as 'Sold Out'. Badges include an auto-expiry timer so stale statuses clear themselves automatically."

### Q6: "Why not use existing security cameras"

**Answer:** “First, security cameras are legally required to store footage for weeks, which completely contradicts our PIPL privacy architecture of instantaneous RAM-based destruction. Second, security cameras are positioned for loss prevention at doors and cashiers, not table-level occupancy.”

## 6. Pre-Launch Validation & Pilot Execution Roadmap

Phase 1: Clearances ⟶ Phase 2: Hardware Setup ⟶ Phase 3: Pilot Testing ⟶ Phase 4: Full Launch
(Months 1-2) (Months 3-4) (Months 5-6) (Months 7-8)

### Phase 1: Clearances & Regulatory Filings (Months 1–2)

1. Submit PIPIA documentation proving in-memory frame destruction to the **Canteen Management Center (后勤处)** and **Public Security Office (保卫处)**.
2. Coordinate with the **Network Center (信息化技术中心)** for static local IP allocation and internal VPN jump-host configuration.
3. Mount physical privacy notices at all canteen entrances.

## Phase 2: Hardware Setup & Infrastructure Wiring (Months 3–4)

1. Install 8 top-down dome PoE cameras over seating zones in the pilot canteen wing.
2. Mount the sealed fanless industrial PC and 8-port PoE switch inside a locked communications cabinet.
3. Complete single-drop Cat6 wiring and confirm dual-NIC subnet isolation.

## Phase 3: Software Deployment & Pilot Validation (Months 5–6)

1. Deploy headless Ubuntu 22.04 LTS, OpenVINO runtime libraries, and the table-polygon calibration tool.
2. Conduct a **2-Week Peak-Hour Ground-Truth Trial** (11:30 AM – 12:30 PM):
    - Compare system output against manual ground-truth seat counts across 6 test tables.
    - Target Benchmark: >90% seat-state accuracy under lunch-rush lighting conditions.
3. Validate heartbeat fail-safe routines and automated night-time Laplacian lens sharpness evaluations.

## Phase 4: WeChat MiniProgram Launch & Rollout (Months 7–8)

1. Publish the MiniProgram client under the official university WeChat enterprise account.
2. Launch the 3-layer MiniProgram interface with bilingual (Chinese/English) support.
3. Review initial usage metrics and scale hardware deployment to remaining canteen floors.